ATIVIDADE PRÁTICA — ATRIBUTOS E RELACIONAMENTOS

Questão 1

Considere a classe:

```
class Produto:


    def __init__(self, codigo, nome, preco):
        self.codigo = codigo
        self.nome = nome
        self.preco = preco
```

Identifique:

a) nome da classe;

Produto

b) atributos de instância;

Codigo, nome, preco

c) método construtor;

__init__

d) função do parâmetro self.

O parâmetro self serve para representar e referenciar a instância tual de uma classe, permitindo acessar seus próprios atributors e métodos.

Questão 2

Analise:

```
class Aluno:
```

```
    instituicao = "Faculdade ABC"

    def __init__(self, nome, matricula):
        self.nome = nome
        self.matricula = matricula
```

Classifique:

instituicao: atributo de classe

nome: atributo de instância

matricula: atributo de instância

como atributo de classe ou atributo de instância.

Questão 3

Crie uma classe Livro contendo:

codigo

titulo

autor

ano

Crie dois objetos diferentes e exiba seus dados.

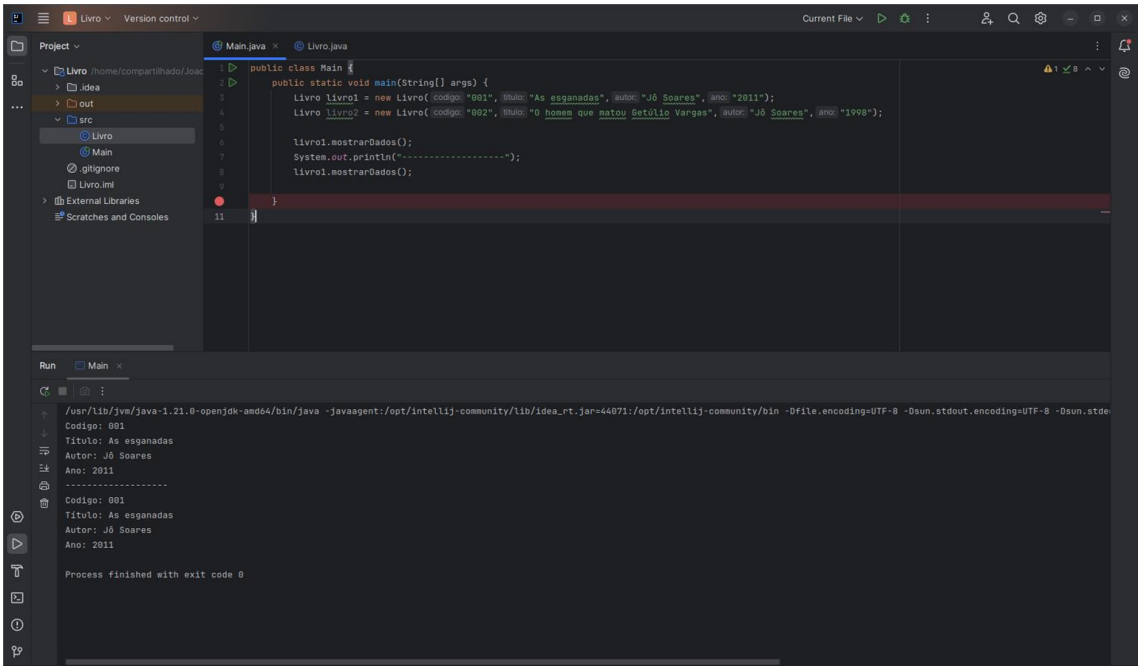
Main.java Livro.java

```java
public class Livro {  no usages
    private String codigo;  2 usages
    private String titulo;  2 usages
    private String autor;  2 usages
    private String ano;  2 usages

    public Livro(String codigo, String titulo, String autor, String ano){  no usages
        this.codigo = codigo;
        this.titulo=titulo;
        this.autor=autor;
        this.ano=ano;
    }

    public void mostrarDados(){  no usages
        System.out.println("Codigo: "+this.codigo);
        System.out.println("Titulo: "+this.titulo);
        System.out.println("Autor: "+this.autor);
        System.out.println("Ano: "+this.ano);
    }


}
```



Questão 4

Considere:

Cliente possui vários Pedidos.

Determine:

a) as classes envolvidas;

R.: Existe a classe Pedidos e a classe Cliente.

b) a cardinalidade;

R.: A cardinalidade é 1:N e N:1, pois um cliente possui vários pedidos e vários pedidos podem ter somente um cliente.]

c) qual classe deverá possuir uma coleção de objetos.

R.: A classe Clientes deve possuir a coleção de objetos de classe Pedido para poder armazenar todos os pedidos que o cliente fez.

Questão 5

Implemente as classes:

Departamento

Professor

Um departamento poderá possuir vários professores.

A classe Departamento deverá possuir:

adicionar_professor()

listar_professores()

Departamento.java

```java
import java.util.ArrayList;

public class Departamento {  2 usages
    private String nome;  1 usage
    private ArrayList<Professor> professores = new ArrayList<~>();  3 usages

    public Departamento(String nome) { this.nome = nome; }

    public void adicionar_professor(Professor professor){  2 usages
        this.professores.add(professor);
        System.out.println("Professor Adicionado\n");
    }

    public void listar_professores(){  2 usages

        if(professores.isEmpty()){
            System.out.println("Esse departamento não possui professores\n");
            return;
        }

        System.out.println("Lista de Professores: ");

        for(Professor professor:professores){
            professor.mostrar_dados();
        }
    }
}

```

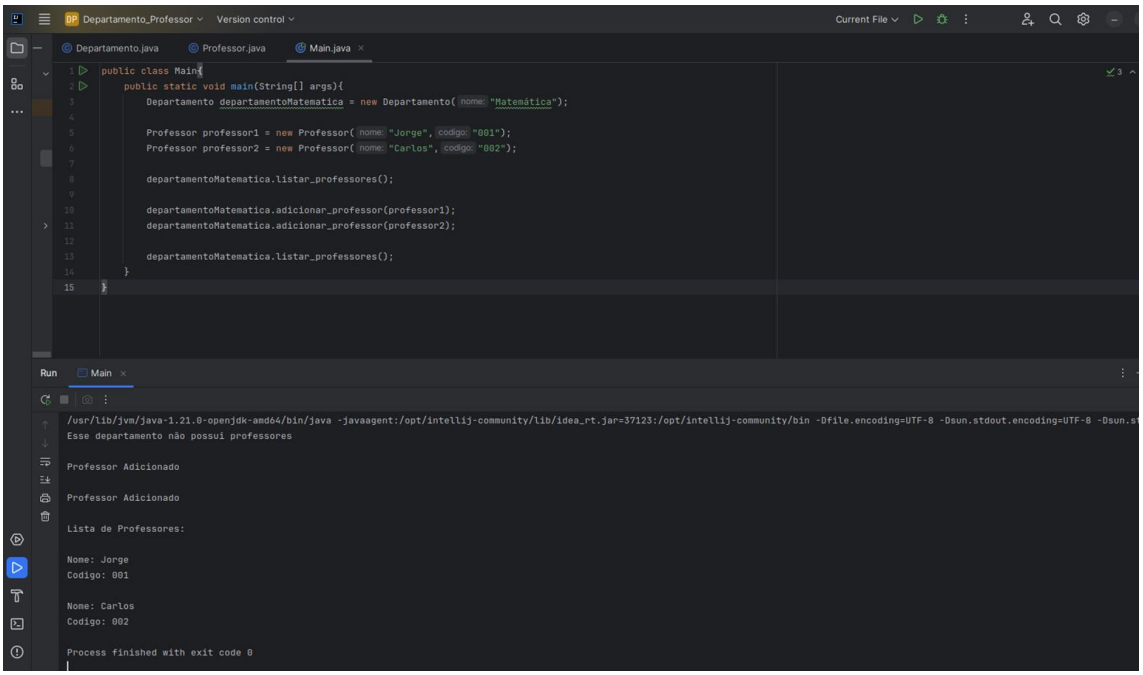
Departamento.java | Professor.java

```java
public class Professor {  8 usages
    private String nome;  2 usages
    private String codigo;  2 usages

    public Professor(String nome, String codigo) {  2 usages
        this.nome = nome;
        this.codigo = codigo;
    }

    public void mostrar_dados(){  1 usage
        System.out.println(
                "\nNome: "+nome+
                "\nCodigo: "+codigo
        );
    }
}

```



Questão 6

Analise:

Pedido possui vários ItemPedido.

Explique por que ItemPedido pode ser modelado utilizando composição em relação a Pedido.

R.: Existem algumas razões, dentre elas pode-se elencar:

- O ItemPedido só existe quando um Pedido existir, ou seja sua existência está atrelada a do Pedido.
- A cardinalidade, um Pedido pode ter vários itens, porém um item só pode ser atribuído a um pedido.
- O Pedido é responsável pela existência dos itens, se ele for excluído todos os itens também são.

Questão 7

Considere:

Aluno cursa Disciplina.

Sabendo que um aluno pode cursar várias disciplinas e uma disciplina pode possuir vários alunos:

a) determine a cardinalidade;

R.: A cardinalidade é de M:N.

b) represente conceitualmente o relacionamento;

Aluno(1) ↔ (0..*)Matrícula

Disciplina(1) ↔ (0..*)Matrícula

c) explique por que listas podem ser necessárias nas duas classes.

R.: Listas são necessárias nas duas classes para armazenar dados de ambas as classes, na do Aluno é necessária para catalogar todas as disciplinas as quais ele está matriculado, e; na de Disciplinas é importante para catalogar todos os alunos que estão inscritos na disciplina.

Questão 8

Um sistema acadêmico precisa armazenar:

Aluno

Disciplina

nota

frequencia

data_matricula

Explique por que nota, frequencia e data_matricula podem ser atributos de uma classe Matricula, em vez de atributos diretamente de Aluno ou Disciplina.

R.: Colocando esses atributos em uma classe matrícula é possível reduzir a duplicidade de informações entre as classes de Aluno e Disciplina, diminuindo a memória gasta no programa e melhorando a eficiência dele.

Questão 9

Desenvolva as classes:

Cliente

Pedido

Requisitos:

Cliente

nome

cpf

pedidos

Pedido

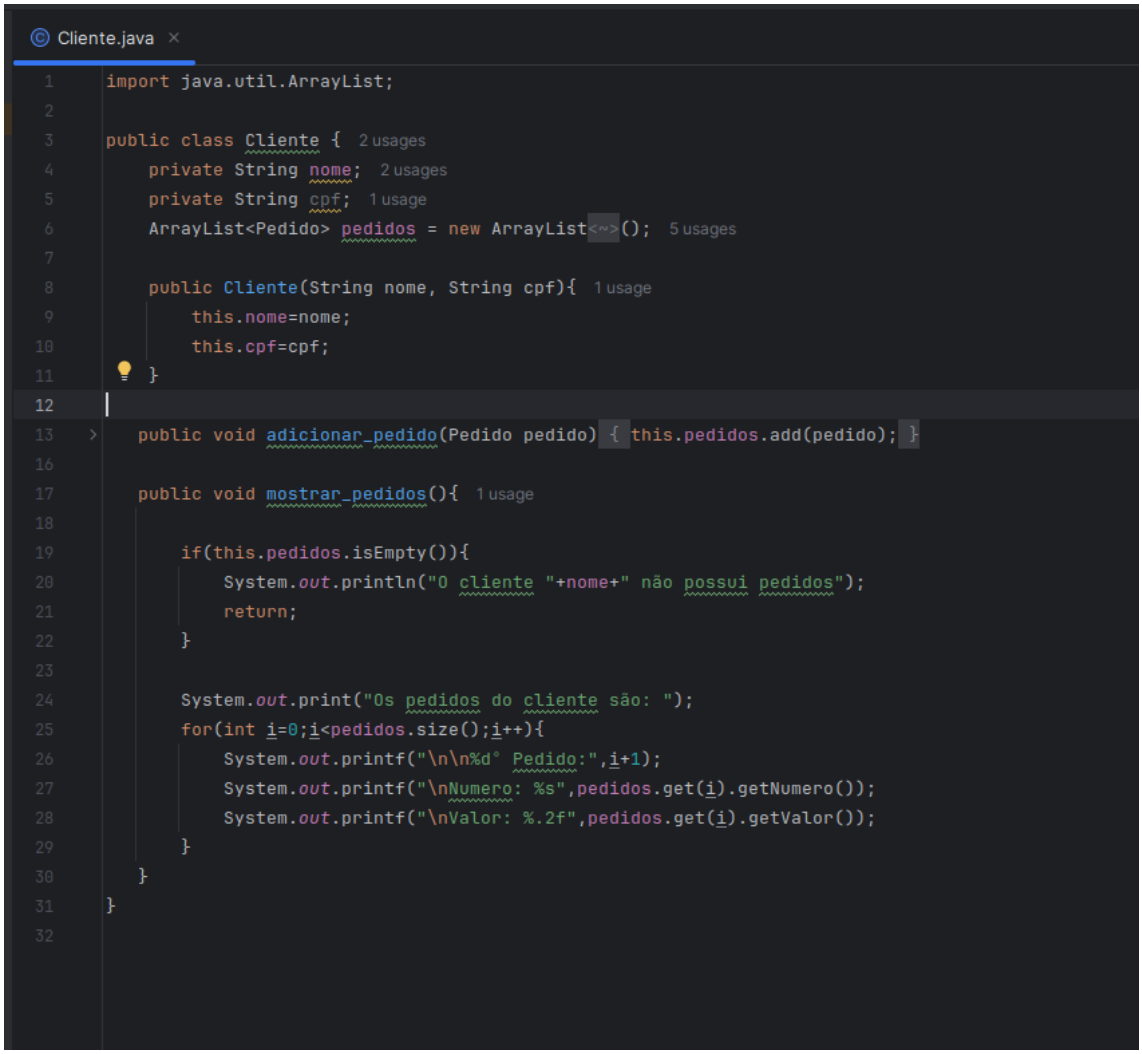
numero

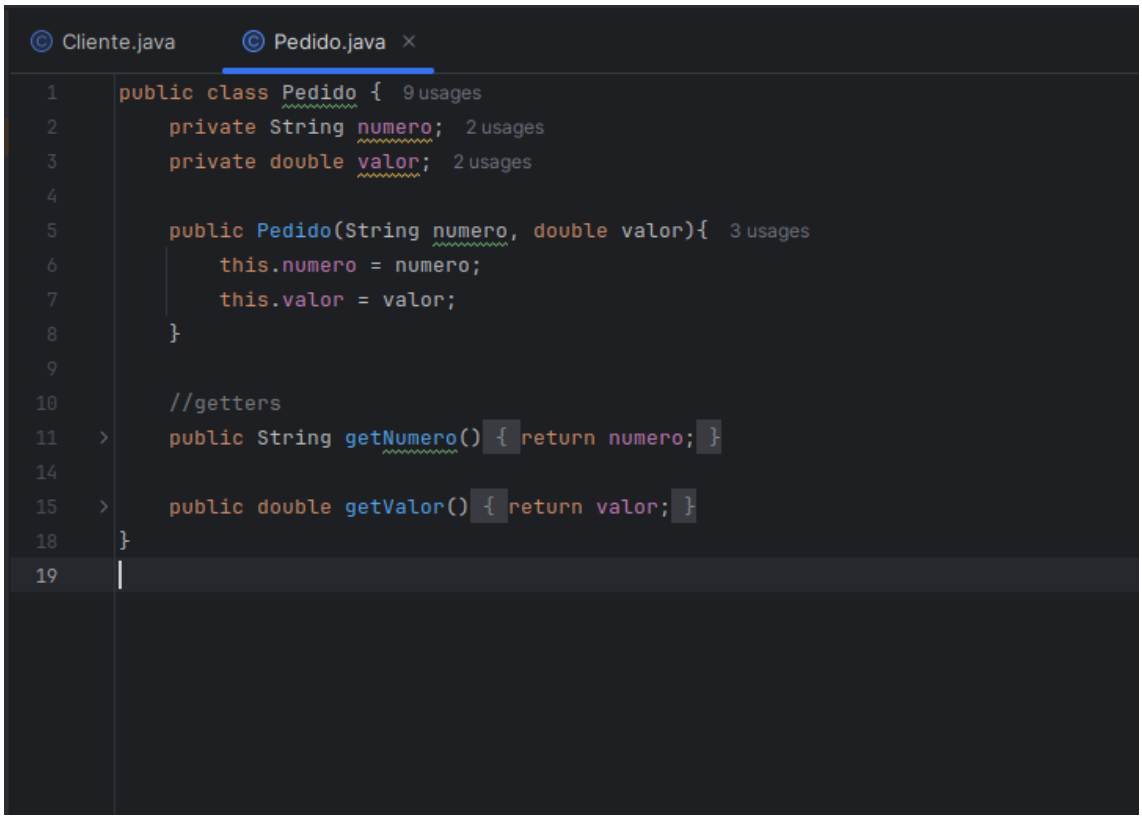
valor

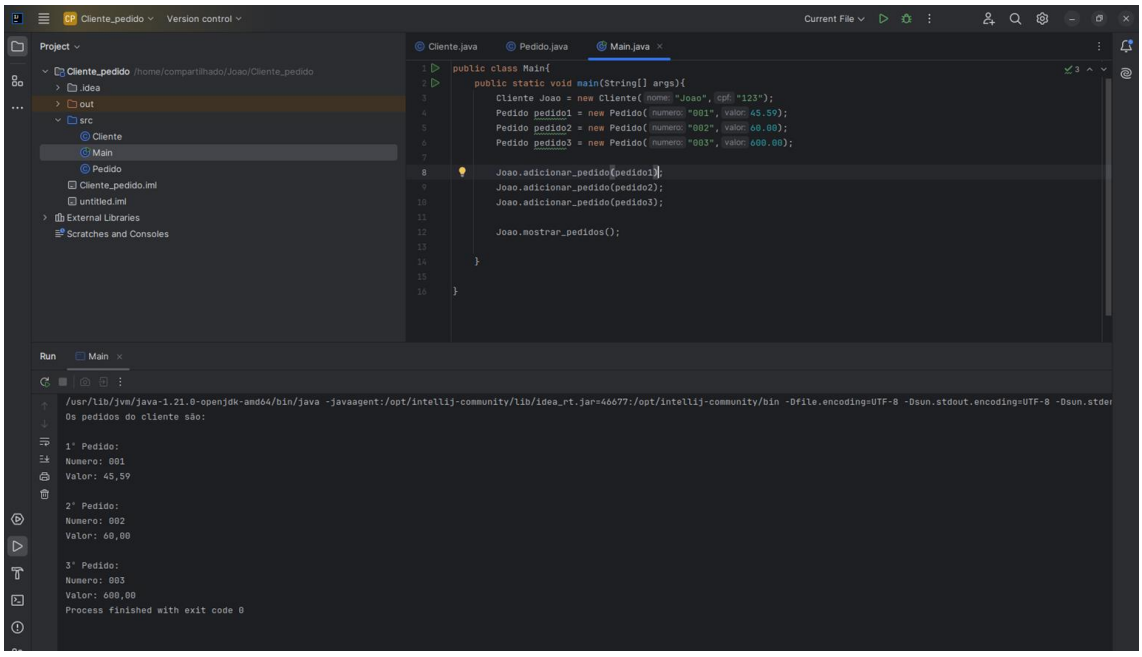
O cliente deverá possuir o método:

adicionar_pedido()

Crie um cliente com três pedidos e exiba todos os pedidos.







Questão 10 — DESAFIO

Desenvolva um pequeno sistema orientado a objetos para uma biblioteca.

Classes obrigatórias:

Livro

Usuario

Emprestimo

Livro

Atributos:

codigo

titulo

autor

Usuario

Atributos:

matricula

nome

Emprestimo

Atributos:

usuario

livro

data_emprestimo

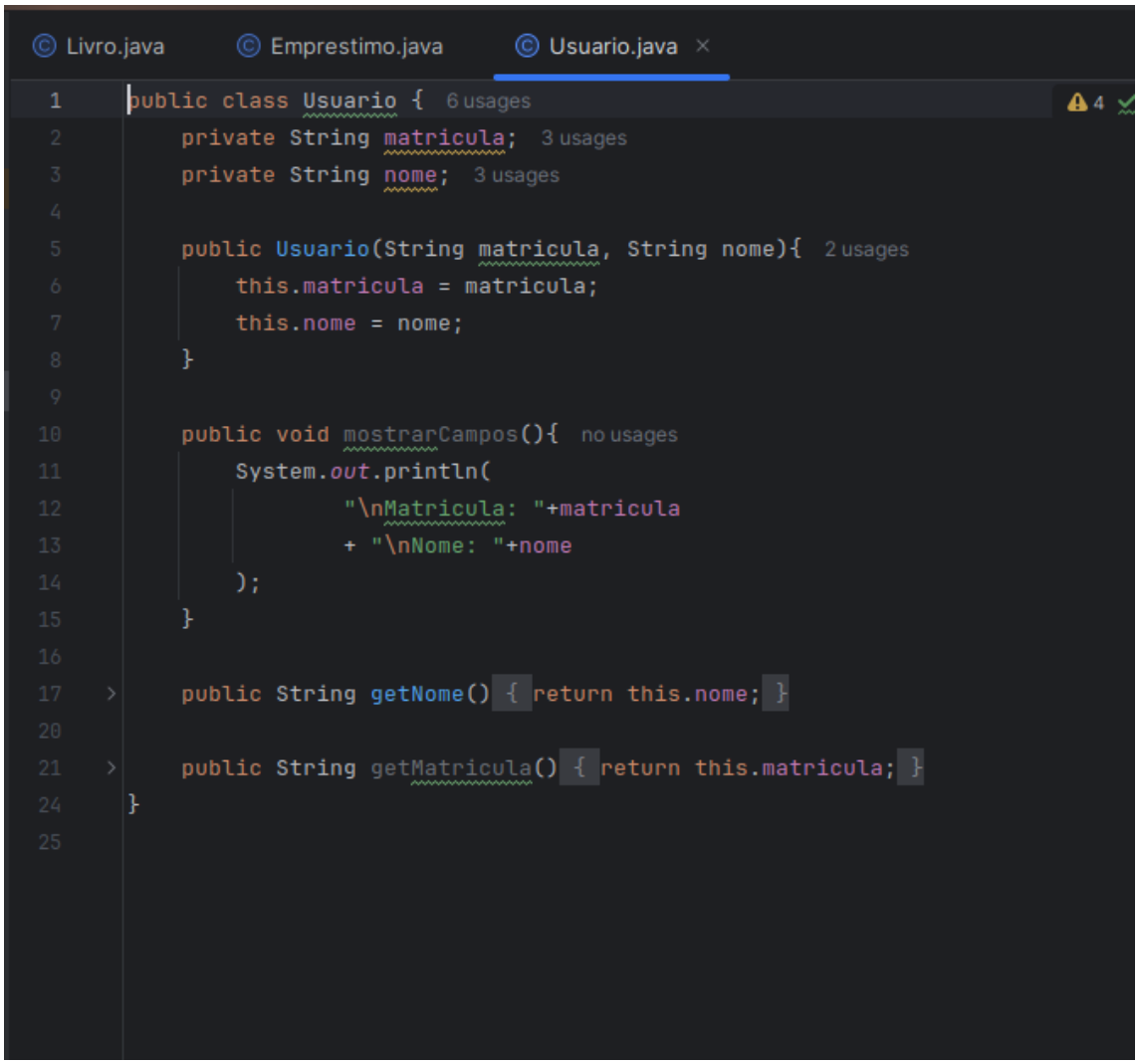
O programa deverá:

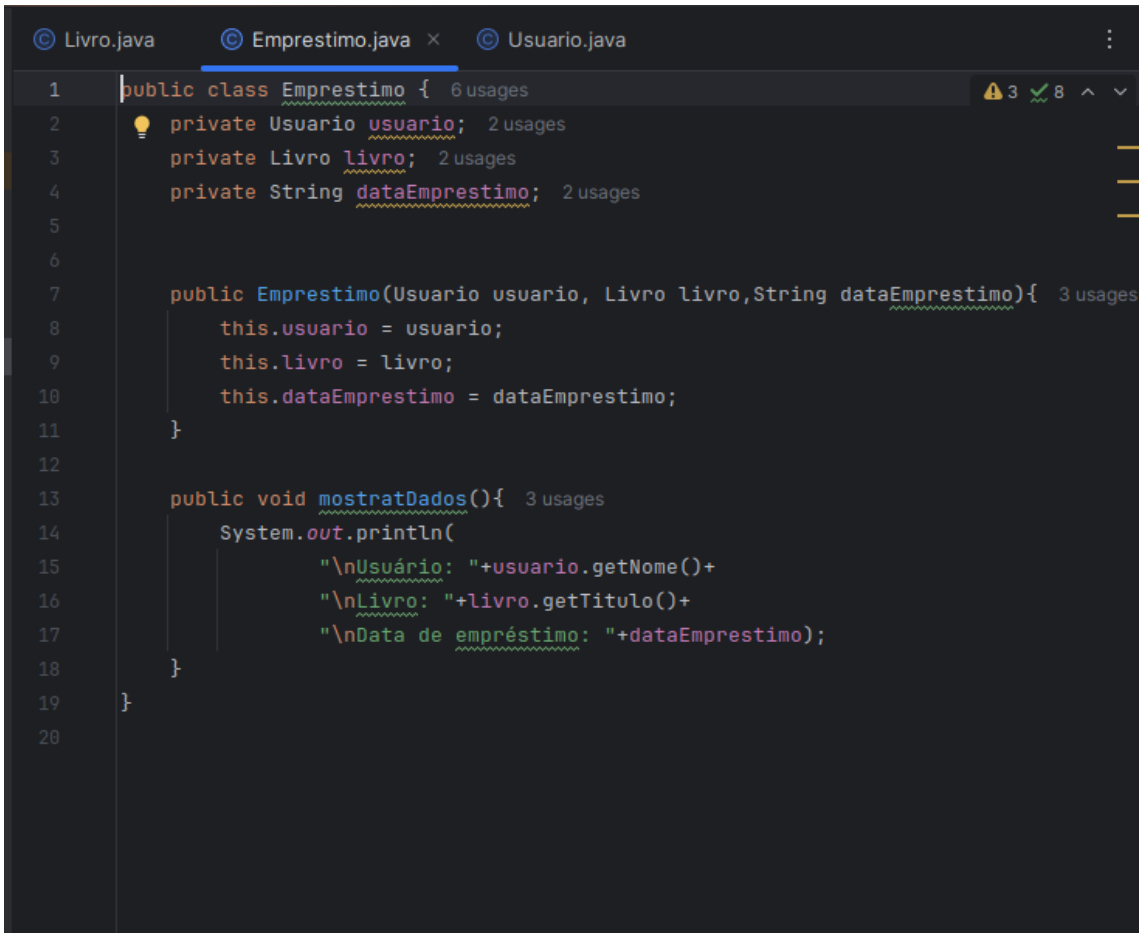
1. criar dois livros;
2. criar dois usuários;
3. registrar pelo menos três empréstimos;
4. relacionar corretamente usuário e livro;
5. exibir:

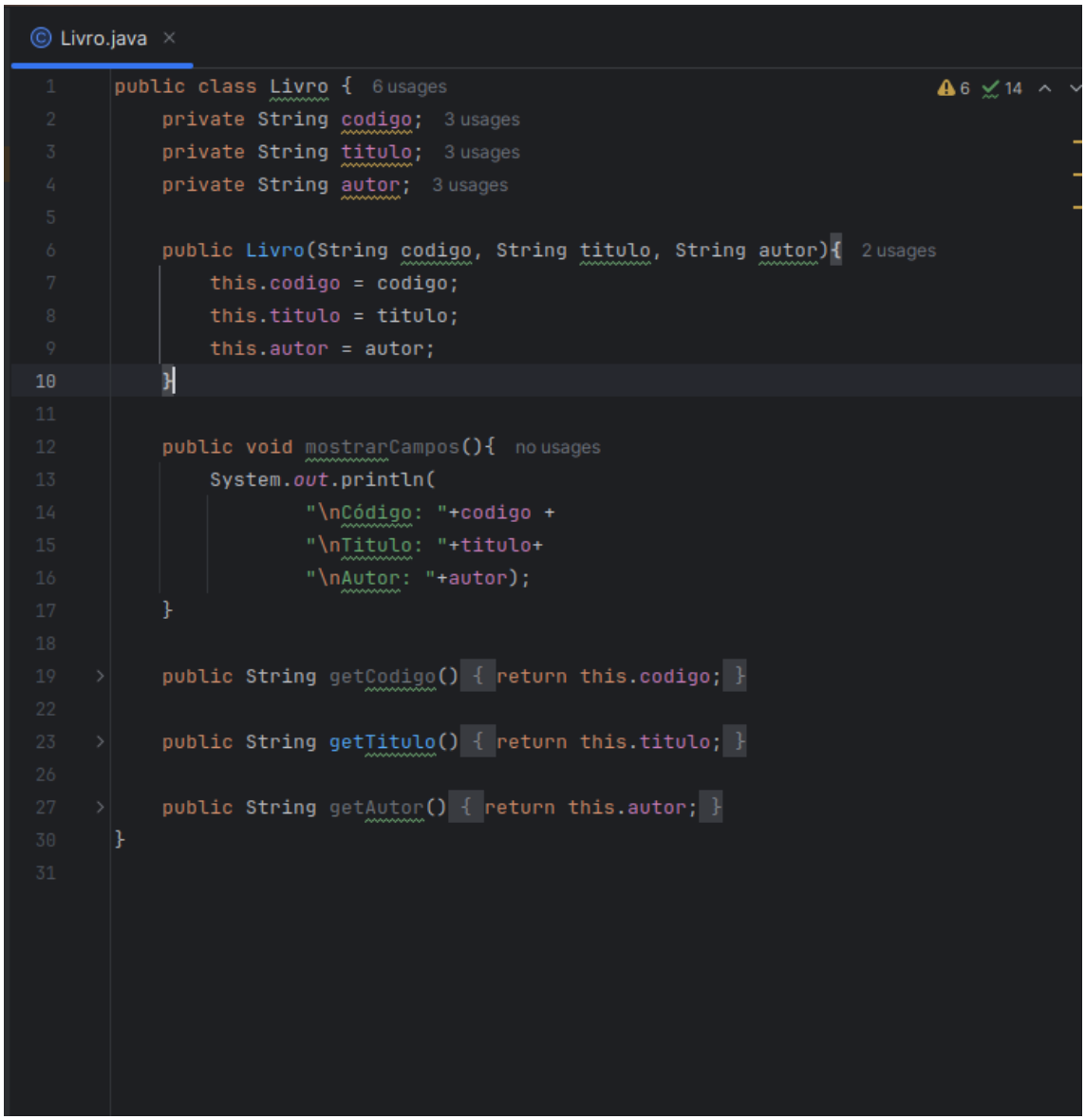
Usuário

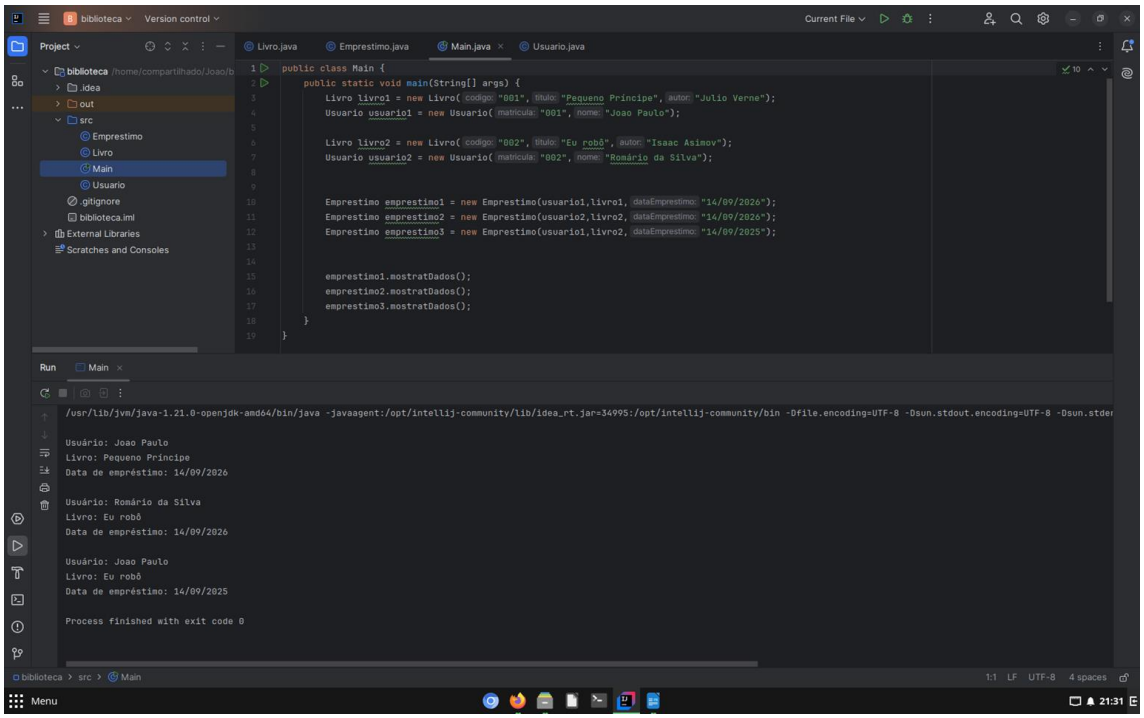
Livro

Data do empréstimo


Livro.java
Emprestimo.java
Usuario.java
public class Usuario { 6 usages
4
private String matricula; 3 usages
private String nome; 3 usages
public Usuario(String matricula, String nome){ 2 usages
this.matricula = matricula;
this.nome = nome;
}
public void mostrarCampos(){ no usages
System.out.println(
"\nMatricula: "+matricula
+ "\nNome: "+nome
);
}
public String getNome() { return this.nome; }
public String getMatricula() { return this.matricula; }
}


Livro.java
Emprestimo.java
Usuario.java
public class Emprestimo { 6 usages
3 8
private Usuario usuario; 2 usages
private Livro livro; 2 usages
private String dataEmprestimo; 2 usages
public Emprestimo(Usuario usuario, Livro livro,String dataEmprestimo){ 3 usages
this.usuario = usuario;
this.livro = livro;
this.dataEmprestimo = dataEmprestimo;
}
public void mostratDados(){ 3 usages
System.out.println(
"\nUsuário: "+usuario.getNome()+
"\nLivro: "+livro.getTitulo()+
"\nData de empréstimo: "+dataEmprestimo);
}
}


Livro.java
public class Livro { 6 usages
6 14
private String codigo; 3 usages
private String titulo; 3 usages
private String autor; 3 usages
public Livro(String codigo, String titulo, String autor){ 2 usages
this.codigo = codigo;
this.titulo = titulo;
this.autor = autor;
}
public void mostrarCampos(){ no usages
System.out.println(
"\nCódigo: "+codigo +
"\nTitulo: "+titulo+
"\nAutor: "+autor);
}
public String getCodigo() { return this.codigo; }
public String getTitulo() { return this.titulo; }
public String getAutor() { return this.autor; }
}


biblioteca
Version control
Current File
Project
Livro.java
Emprestimo.java
Main.java
Usuario.java
biblioteca /home/compartilhado/Joao/b
.idea
out
src
Emprestimo
Livro
Main
Usuario
.gitignore
biblioteca.iml
External Libraries
Scratches and Consoles
public class Main {
public static void main(String[] args) {
Livro livro1 = new Livro(codigo: "001", titulo: "Pequeno Principe", autor: "Julio Verne");
Usuario usuario1 = new Usuario(matricula: "001", nome: "Joao Paulo");
Livro livro2 = new Livro(codigo: "002", titulo: "Eu robô", autor: "Isaac Asimov");
Usuario usuario2 = new Usuario(matricula: "002", nome: "Romário da Silva");
Emprestimo emprestimo1 = new Emprestimo(usuario1,livro1, dataEmprestimo: "14/09/2026");
Emprestimo emprestimo2 = new Emprestimo(usuario2,livro2, dataEmprestimo: "14/09/2026");
Emprestimo emprestimo3 = new Emprestimo(usuario1,livro2, dataEmprestimo: "14/09/2025");
emprestimo1.mostratDados();
emprestimo2.mostratDados();
emprestimo3.mostratDados();
}
}
Run
Main
/usr/lib/jvm/java-1.21.0-openjdk-amd64/bin/java -javaagent:/opt/intellij-community/lib/idea_rt.jar=34995:/opt/intellij-community/bin -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stder
Usuário: Joao Paulo
Livro: Pequeno Principe
Data de empréstimo: 14/09/2026
Usuário: Romário da Silva
Livro: Eu robô
Data de empréstimo: 14/09/2026
Usuário: Joao Paulo
Livro: Eu robô
Data de empréstimo: 14/09/2025
Process finished with exit code 0
biblioteca > src > Main
1:1 LF UTF-8 4 spaces
Menu
21:31